

Relatório de Tentativas de Solução — Extração Estruturada

Projeto: Assistente de Editais (TCC) — Branch develop

Data: 15/04/2026

1. Contexto

O projeto possui um pipeline de ingestão de editais de concursos públicos: extração de texto via pdfplumber, extração estruturada via LLM (JSON validado por Pydantic), persistência em SQLite e indexação vetorial em ChromaDB. O laudo do projeto documenta dois problemas na extração estruturada: omissão seletiva (o LLM retorna fração dos itens — 4 de 13 blocos de conteúdo programático) e paráfrase distorcida (o LLM inventa restrições inexistentes no documento). Foram formuladas duas hipóteses concorrentes: H2 (filtragem por domínio antes da extração) e H1 (extração incremental página por página). Ambas foram tentadas e falharam conforme descrito abaixo.

2. Tentativa 1 — H2: Filtro de Domínio por Página

Abordagem: filtro por domínio, conceito e implementação

Criado o módulo `ingestion/filtro_dominio.py`. O LLM recebia o texto de cada página (iterando as 47 páginas do edital) junto com um contexto de leitura passado entre páginas, e devolvia o texto filtrado (mantido ou descartado) e o contexto atualizado. A instrução era manter conteúdo relevante para Ciência de Dados e descartar conteúdo exclusivo de outras áreas. A saída filtrada substituiria o texto completo no restante do pipeline.

Resultado

O filtro descartou páginas 5 a 12 inteiras — trecho que contém regras gerais de vagas reservadas (PcD, pessoas negras) e requisitos de investidura, conteúdo obrigatório para qualquer candidato. O LLM interpretou menções a outras profissões (ex: Psicologia no topo da página 4) como sinal para descartar tudo que veio em seguida. Adicionalmente, a página 4 falhou o parse de JSON nas duas tentativas (texto com caracteres especiais dentro de string JSON). O fallback manteve a página, mas evidenciou a fragilidade do formato de saída.

Causa raiz

Dois problemas estruturais: (1) o prompt tratava manter e descartar como decisões equivalentes, quando na realidade a grande maioria de um edital é conteúdo geral — o descarte deveria ser a exceção rara; (2) pedir ao LLM para copiar o texto inteiro da página dentro de uma string JSON é inerentemente frágil (aspas, quebras de linha, caracteres especiais corrompem o JSON) e caro em tokens (o LLM regenera token por token o que já temos). Ajustes no prompt foram cogitados mas descartados — o problema era de design, não de instrução.

3. Tentativa 2 — H1: Extração Incremental Página por Página

Abordagem: extração estruturada incremental

Reescrito o módulo `ingestion/structured_extract.py`. Em vez de mandar o texto inteiro ao LLM em 2-3 chamadas (abordagem original), o texto era processado página por página (47 chamadas). Cada chamada recebia: o JSON completo do EditalInfo com o estado atual do preenchimento, um contexto de leitura (seção atual, conteúdo em andamento), e o texto da página. O LLM devolvia o JSON inteiro atualizado e o contexto atualizado.

Resultado

A extração levou aproximadamente 20 minutos (47 chamadas ao gpt-4o-mini, ~25 segundos cada). Ao final, o Pydantic rejeitou o JSON com erros de validação: o campo 'orgao' (obrigatório) havia desaparecido e o campo 'cronograma' havia mudado de lista para dicionário. O LLM distorceu progressivamente a estrutura do JSON ao longo das 47 reescritas.

Causa raiz

Três problemas: (1) o prompt não instruíra foco em Ciência de Dados — o LLM extraía todas as ênfases de todas as áreas, inflando o JSON desnecessariamente; (2) o JSON crescia a cada iteração e era reenviado inteiro, gerando alto custo de tokens e latência crescente; (3) após 47 reescritas do JSON completo, o LLM apresentou drift estrutural — campos obrigatórios sumiram e tipos de dados mudaram (lista virou dict). A abordagem de reescrever o JSON inteiro a cada página é inerentemente instável para documentos longos.

4. Lições e Direção Futura

Ambas as tentativas falharam por razões complementares. A H2 falhou porque o LLM não tem visão do documento inteiro ao decidir página por página, e o formato de saída (texto dentro de JSON) é frágil. A H1 falhou porque 47 reescritas do JSON inteiro causam drift estrutural, e a ausência de foco em Ciência de Dados tornava a tarefa desnecessariamente grande. Uma direção promissora discutida mas não implementada é um agente de ingestão que: percorre o documento mantendo contexto contínuo, constrói um índice de tópicos (onde começam, terminam, do que tratam), classifica cada tópico por relevância para Ciência de Dados, e realiza a extração estruturada por tópico — não por página. Esse índice serviria também ao agente de resposta como mapa para consultas direcionadas.